SecretHandle: OS-Minted, Destination- and Task-Bound Opaque Secrets for Android Mobile AI Agents

Author Name(s) TBD
Affiliation TBD
City, Country
email@example.com

Abstract

Mobile GUI agents must often use a one-time password, credential, address, phone number, or payment identifier to finish a task. Today the value is placed in a screenshot, accessibility tree, prompt, or action argument, so an injected instruction can convert task authority into disclosure or redirection. We present SecretHandle, an Android design for *usable but invisible* secrets: a privileged service mints an opaque, authenticated handle at the authoritative OS source—the SMS or notification OTP pipeline, Credential Manager, the clipboard, or a wallet—before the raw value can reach the agent. The handle binds a secret type, a task identifier, a destination predicate over package, WebView origin, and field class, and a type-specific lifecycle (single-use and TTL for OTPs, task-scoped for identifiers, confirm-per-use for payment). The agent observes only the handle and issues `ENTER(handle)` toward a UI node; the OS resolves the value only after authenticating the handle, validating task and target, enforcing expiry/use/confirmation, and reserving the handle against races, then redacts the resolved field from subsequent agent observation. We report an on-emulator reproduction of the OTP-forwarding attack and a labeled, *simulation-only* reference-model study whose trends motivate the design; they are not device measurements and are not presented as paper results. We close with a preregistration-style evaluation plan (killer experiment, attack families, and baselines) for the full system.

CCS Concepts

• **Security and privacy** → **Mobile platform security**; *Authorization*; • **Human-centered computing** → Ubiquitous and mobile computing.

Keywords

mobile AI agents, Android security, opaque capabilities, destination binding, one-time passwords, prompt injection, observation redaction

1 Introduction

Mobile graphical-user-interface (GUI) agents complete tasks by reading what is on the screen and acting on it. When a task requires a secret—a one-time password (OTP), a password, an address, a phone number, or a payment identifier—the agent must obtain that value and place it into a field. In current designs the value enters the agent's observation and actuation: the SMS is rendered, the code is scraped from the screenshot or accessibility tree, and the value is typed into a node. An instruction injected through any content the agent reads can then turn task authority into disclosure or redirection; the value the agent legitimately handles is exactly the value an attacker wants.

This is no longer hypothetical. A browser agent read an OTP from a mail client and published it; mobile agents have forwarded Alipay verification codes under injected instructions (see Section 2 for the primary sources). Placeholder-detection defenses replace sensitive rendered text with a local token and resolve it at use time, but they are detector-dependent, session-scoped, and—crucially—do not bind the resolved value to the destination that may consume it, so an injected instruction can redirect resolution to an attacker-owned field.

Research question. Can Android let an agent *use* a secret without ever *observing* it, by minting unforgeable opaque handles at the authoritative OS source and resolving a handle only into a destination the handle is bound to—so that neither the raw value nor a usable copy enters the agent's perception or a wrong destination?

Why mobile. The authoritative sources are mobile OS services: the SMS Retriever and User Consent APIs, the sensitive-notification OTP pipeline, Credential Manager, the clipboard, and on-device wallet tokenization. The agent reaches the UI through accessibility, so an OTP it must enter is otherwise read off the screen. Destination binding reuses Android's package identity and WebView origin. On a server there is no OS SMS pipeline, no Credential Manager, no on-device wallet, and no UI to type into; the problem and its enforcement point are mobile.

Contribution. We propose SecretHandle, an OS layer that makes secrets usable but invisible to untrusted agents. A handle is minted at the source before the raw value reaches the agent, carries a destination-and-task binding, and has a typed lifecycle. A resolver injects the value only into a bound destination; a redactor masks the resolved field afterwards. This emerging-ideas paper contributes (i) the problem framing and threat model, (ii) the handle/resolver/redactor design with an unforgeable, destination- and task-bound abstraction, (iii) an on-emulator reproduction of the motivating attack and a clearly labeled simulation-only reference-model study, and (iv) a preregistration-style evaluation plan for the full system. We do *not* report device measurements in this paper.

2 Background and Motivation

Mobile agents act through the UI.. Agents such as AppAgent and the Mobile-Agent family drive Android through the accessibility tree, screenshots, and synthesized gestures [23, 26, 29]. Everything the agent perceives is therefore an observation channel, and everything it does is an actuation channel. A secret that must be entered crosses both.

The motivating attack. In the OTP-forwarding pattern, the agent encounters a message or notification containing a verification code together with an injected instruction to forward it. The agent reads the code from the rendered text and sends it to an attacker-controlled endpoint [16, 17]. We reproduced the core mechanism on an Android emulator with an open notification-listener agent: the code was read from the notification and POSTed to an isolated endpoint (Section 5). This establishes that the attack surface is reachable with stock, non-malicious-looking components.

Why existing defenses are insufficient. Placeholder anonymization replaces rendered sensitive text with a local token and resolves it at `type` time. It removes the literal value from the current observation but depends on detector recall, is session-scoped, and does not constrain *which* field receives the resolved value or *which* origin may request it. An injected instruction can redirect the resolution [24, 30]. *Platform autofill and Credential Manager* are app-initiated: the consuming app must drive them, and they trust that app, so an accessibility agent cannot invoke them and a hostile field is not distinguished [1, 3, 4]. *Server-side agentic-payment tokens* cover payments only and live outside the OS, leaving OTP, phone, email, address, and clipboard unprotected.

Design forces. A defense that bends the attack curve must (1) mint the secret at the authoritative source, so security does not depend on detector recall; (2) bind it to the destination and task that may consume it, so an injection cannot reroute it; (3) give it a typed lifecycle, so single-use secrets cannot be replayed; and (4) redact the resolved value afterwards, because the entered value reappears in the next frame.

3 Threat Model

Assets. OTPs; account credentials; phone numbers, email addresses, and postal addresses; payment and account identifiers; and any secret whose value is usable if disclosed.

Trusted. The `system_server` and the SecretHandle service; the OS source pipelines (SMS/notification OTP classifier, Credential Manager, clipboard, wallet); and hardware-backed key storage (StrongBox, or a software keystore where hardware is unavailable).

Untrusted. The agent and its model, which may be prompt-injected; third-party applications, one of which may render handle-like strings or send a fake OTP message; cloud model endpoints; and web content reached through an in-app WebView.

Attacker goals. (G1) read a secret the agent handles; (G2) redirect a secret to an attacker-controlled destination; (G3) replay or reuse a secret; (G4) forge a handle.

Security goals. For a task that legitimately requires a secret, SecretHandle aims to guarantee: (S1) the raw value never enters the agent's observation or a non-bound destination; (S2) handles are unforgeable OS capabilities; (S3) resolved values are redacted from subsequent agent observation; (S4) single-use secrets cannot be reused. The utility goal is that login, 2FA, and checkout flows still complete.

Out of scope. Semantic misuse *inside* an allowed destination (e.g., an agent tricked into initiating a legitimate-but-unwanted action) and side channels outside the modeled observation boundary. These are residual risks and are discussed in the evaluation plan, not claimed to be eliminated.

4 Design: Usable but Invisible Secrets

A *secret handle* is an unforgeable capability object minted by the OS at the authoritative source. It carries (i) a secret *type*, (ii) a *destination binding*—an allowed package and, for web, a document origin, plus an allowed field class—derived from the task and, where available, from the source's own binding (e.g., the app hash used by the SMS Retriever [7]), and (iii) a *lifecycle*: single-use with a TTL for OTPs, task-scoped for identifiers, and confirm-per-use for payment. The agent observes only the handle and reasons about it ("the code arrived; enter it into the login field"); it never observes the value.

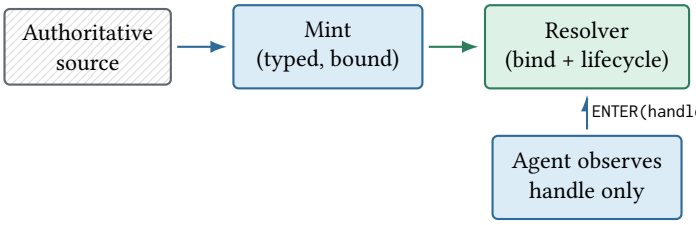


Figure 1: SecretHandle flow: the value stays inside the OS from source to a bound destination; the agent sees only an opaque handle.

Architecture. Four components sit inside the OS:

- **Handle mint** (new system service): hooks the OTP pipeline, Credential Manager, clipboard, and wallet to mint typed, bound, unforgeable handles; keys live in hardware-backed storage.
- **Resolver** (input-dispatch / accessibility action path): on `ENTER(handle)`, authenticates the handle, checks the target node's package/origin/field class against the binding, injects the value, and enforces use count and TTL under a reservation lock.
- **Observation redactor** (accessibility service and the agent's screenshot path): masks resolved values in the agent's view.
- **Agent SDK**: surfaces handles in the observation, provides the `ENTER(handle)` action, and adapts open-source agents.

Resolution as the enforcement point. The security hinges on resolution: a value is injected only if the handle authenticates and the target is inside the binding. An injected instruction that targets an attacker package, a foreign origin, a non-bound field, or a stale/recycled node is rejected, regardless of how convincingly the agent was manipulated. A booking of one use under a lock makes replay and concurrent double-resolution fail. After injection, the redactor prevents the value from reappearing in the next accessibility tree or screenshot.

Constructive building blocks. Destination binding reuses Android package identity and WebView origin; unforgeability uses a message authenticator [18] over a canonical encoding of handle fields with a key separated from the value; lifecycle and reservation are ordinary OS state transitions. The main engineering is wiring the mint points and the resolver into the input path; the design composes with blind-actuation approaches (the agent acts on a node it cannot read).

5 Preliminary Evidence and Evaluation Plan

On-emulator attack reproduction. On an Android emulator, an open notification-listener agent read a synthetic OTP from an SMS notification and forwarded it to a loopback endpoint. Across ten independent synthetic canaries, all ten were read and forwarded (observed exposure rate 1.0 with a Wilson 95% interval of [0.72, 1.0]). This is a functional reproduction of the mechanism, not a device result and not a paper number.

Reference-model study (simulation only). To check the qualitative trends the design predicts, we ran the project's Python reference model on synthetic canaries (seed `20261017`, 200 trials per attack, 4800 trials total). The model is a simulation; its numbers are *not* device measurements and are *not* reported as results. Under it, the unprotected baseline exposed the value in every trial; a detected-placeholder baseline removed the literal value from the immediate artifact but still leaked it after entry (post-entry leakage 1.0), illustrating that detection is not protection; and the SecretHandle condition produced zero exposure, zero redirection, zero post-entry leakage, zero forgery and replay acceptance, with full legitimate-task success and zero false rejections. The platform-flow baseline was recorded as not applicable because it could not be exercised. Full configuration, outputs, and caveats are in the project's preliminary-evidence document.

Killer experiment (design of record). The decisive experiment runs the OTP-forwarding injection against an open login-plus-OTP app, first unprotected (the agent forwards the code), then with SecretHandle, where the OTP is a handle bound to the login app's package. It passes only if all three hold: (a) the raw code is absent from every instrumented agent artifact; (b) `ENTER(handle)` into the attacker's destination is rejected by the resolver; and (c) the legitimate login succeeds. In the reference model all three hold; establishing them on a real device is full-paper work.

Planned evaluation. The full paper evaluates four conditions—unprotected, detected placeholder, platform flow, and SecretHandle—against attack families: read-and-exfiltrate; destination redirection; fake handle; replay/race; post-entry reread; same-destination semantic misuse; and identity ambiguity. Metrics are exposure E, redirection R, utility U, post-entry leakage L, forgery acceptance F, and replay acceptance Y, with policy-rejection and false-rejection rates and latency. Binary outcomes use Wilson intervals; paired comparisons use a paired permutation or exact McNemar test; sample sizes are fixed by a prospective precision analysis. Exclusions (crash, outage, cancellation, unsupported identity) are reported by category. General-task regression is measured on the AndroidWorld suite [20], with SPA-Bench [11] for additional breadth. Device latency, energy, and hardware-keystore behavior are explicitly deferred and reported only where a suitable device is available.

6 Related Work

Placeholder and anonymization defenses. Mobile placeholder schemes replace sensitive rendered text with a type-preserving token and reverse the mapping at use time [24, 30]; the closest recent work resolves a `type(PLACEHOLDER)` locally but is detector-based, session-scoped, and validates index ranges rather than which field receives a resolved value. We differ on mint source (authoritative OS source, not screen scraping), unforgeability, destination-and-task binding, typed lifecycle, and a threat model that includes redirection.

Capability and isolation approaches. Planner-side capability schemes and dual-model patterns hide values from the planner in tool-calling settings [13, 14]; isolation and privilege-minimization designs restrict what a tool agent may do [9, 21, 25]. These operate in the planning/tool layer rather than enforcing at the point where a secret crosses from an OS source into a UI field.

Prompt sanitizers. Detection-and-rewrite approaches suppress or restore sensitive spans in prompts [12, 22, 27]. They reduce model-side exposure but do not bind a resolved value to a destination and are not OS-enforced.

Platform and industry mechanisms. The SMS Retriever and User Consent APIs [6, 7], the sensitive-notification OTP pipeline [2], Credential Manager [1], autofill [3, 4], the clipboard sensitivity flag [5], and Protected Confirmation [8] are building blocks, but autofill and Credential Manager are app-initiated and trust the app, and server-side agentic-payment tokens are payment-only. SecretHandle composes these and adds OS-level destination binding and lifecycle.

Attacks. Documented agent OTP exfiltration and forwarding, plus environmental and fine-print injection, motivate the threat model [10, 15–17, 19, 28]. We adapt attack mechanics, not their reported success numbers.

7 Conclusion

SecretHandle reframes how an untrusted mobile agent handles secrets: instead of reading a value and typing it, the agent receives an opaque, unforgeable, destination- and task-bound handle and asks the OS to resolve it into a bound field. The raw value never enters the agent's observation, cannot be rerouted by injection, cannot be replayed past its lifecycle, and is redacted after entry. We have framed the problem, stated the threat model, described the design, reproduced the motivating attack on an emulator, and specified a reproducible evaluation for the full system. The next step is to implement the mint points and resolver in AOSP and run the killer experiment on a real device.

References

[1] Android Developers. 2026. About Credential Manager. https://developer.android.com/identity/credential-manager Accessed 2026-10-03.

[2] Android Developers. 2026. Android 15 Behavior Changes: Security and Privacy. https://developer.android.com/about/versions/15/behavior-changes-all Accessed 2026-10-03.

[3] Android Developers. 2026. Autofill Framework. https://developer.android.com/identity/autofill Accessed 2026-10-03.

[4] Android Developers. 2026. Build Autofill Services. https://developer.android.com/identity/autofill/autofill-services Accessed 2026-10-03.

[5] Android Developers. 2026. ClipDescription.EXTRA_IS_SENSITIVE API Reference. https://developer.android.com/reference/android/content/ClipDescription#EXTRA_IS_SENSITIVE Accessed 2026-10-03.

[6] Android Developers. 2026. One-Tap SMS Verification with the SMS User Consent API. https://developers.google.com/identity/sms-retriever/user-consent/overview Accessed 2026-10-03.

[7] Android Developers. 2026. Request SMS Verification in an Android App. https://developer.android.com/identity/sms-retriever Accessed 2026-10-03.

[8] Android Open Source Project. 2026. Android Protected Confirmation. https://source.android.com/docs/security/features/protected-confirmation Accessed 2026-10-03.

[9] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. 2024. AirGapAgent: Protecting Privacy-Conscious Conversational Agents. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 3868–3882. doi:10.1145/3658644.3690350

[10] Chaoran Chen, Zhiping Zhang, Bingcan Guo, Shang Ma, Ibrahim Khalilov, Simret A. Gebreegziabher, Yanfang Ye, Ziang Xiao, Yaxing Yao, Tianshi Li, and Toby Jia-Jun Li. 2025. The Obvious Invisible Threat: LLM-Powered GUI Agents' Vulnerability to Fine-Print Injections. *arXiv preprint arXiv:2504.11281* (2025). doi:10.48550/arXiv.2504.11281

[11] Jingxuan Chen, Derek Yuen, Bin Xie, Yuhao Yang, Gongwei Chen, Zhihao Wu, Li Yixing, Xurui Zhou, Weiwen Liu, Shuai Wang, Kaiwen Zhou, Rui Shao, Liqiang Nie, Yasheng Wang, Jianye Hao, Jun Wang, and Kun Shao. 2024. SPA-Bench: A Comprehensive Benchmark for SmartPhone Agent Evaluation. *arXiv preprint arXiv:2410.15164* (2024). doi:10.48550/arXiv.2410.15164

[12] Chun Jie Chong, Chenxi Hou, Zhihao Yao, and Seyed Mohammadjavad Seyed Talebi. 2024. Casper: Prompt Sanitization for Protecting User Privacy in Web-Based Large Language Models. *arXiv preprint arXiv:2408.07004* (2024). doi:10.48550/arXiv.2408.07004

[13] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. *arXiv preprint arXiv:2505.23643* (2025). doi:10.48550/arXiv.2505.23643

[14] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813* (2025). doi:10.48550/arXiv.2503.18813

[15] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *arXiv preprint arXiv:2406.13352* (2024). doi:10.48550/arXiv.2406.13352

[16] Chenghao Du, Quanfeng Huang, Tingxuan Tang, Zihao Wang, Adwait Nadkarni, and Yue Xiao. 2025. Measuring the Security of Mobile LLM Agents under Adversarial Prompts from Untrusted Third-Party Channels. *arXiv preprint arXiv:2510.27140* (2025). doi:10.48550/arXiv.2510.27140

[17] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *arXiv preprint arXiv:2302.12173* (2023). doi:10.48550/arXiv.2302.12173

[18] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. 1997. *HMAC: Keyed-Hashing for Message Authentication.* Technical Report RFC 2104. RFC Editor. doi:10.17487/RFC2104

[19] Zeyi Liao, Lingbo Mo, Chejian Xu, Mintong Kang, Jiawei Zhang, Chaowei Xiao, Yuan Tian, Bo Li, and Huan Sun. 2024. EIA: Environmental Injection Attack on Generalist Web Agents for Privacy Leakage. *arXiv preprint arXiv:2409.11295* (2024). doi:10.48550/arXiv.2409.11295

[20] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. 2024. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. *arXiv preprint arXiv:2405.14573* (2024). doi:10.48550/arXiv.2405.14573

[21] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. *arXiv preprint arXiv:2504.11703* (2025). doi:10.48550/arXiv.2504.11703

[22] Li Siyan, Vethavikashini Chithrra Raghuram, Omar Khattab, Julia Hirschberg, and Zhou Yu. 2025. PAPILLON: Privacy Preservation from Internet-based and Local Language Model Ensembles. In *Proceedings of NAACL-HLT 2025*. 3371–3390. doi:10.18653/v1/2025.naacl-long.173

[23] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *arXiv preprint arXiv:2401.16158* (2024). doi:10.48550/arXiv.2401.16158

[24] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2023. AutoDroid: LLM-powered Task Automation in Android. *arXiv preprint arXiv:2308.15272* (2023). doi:10.48550/arXiv.2308.15272

[25] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Network and Distributed System Security Symposium*. https://www.ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems/

[26] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. 2025. Mobile-Agent-v3: Fundamental Agents for GUI Automation. *arXiv preprint arXiv:2508.15144* (2025). doi:10.48550/arXiv.2508.15144

[27] Ziqian Zeng, Jianwei Wang, Junyao Yang, Zhengdong Lu, Haoran Li, Huiping Zhuang, and Cen Chen. 2025. PrivacyRestore: Privacy-Preserving Inference in Large Language Models via Privacy Removal and Restoration. In *Proceedings of ACL 2025*. 10821–10855. doi:10.18653/v1/2025.acl-long.532

[28] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. *arXiv preprint arXiv:2403.02691* (2024). doi:10.48550/arXiv.2403.02691

[29] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2023. AppAgent: Multimodal Agents as Smartphone Users. *arXiv preprint arXiv:2312.13771* (2023). doi:10.48550/arXiv.2312.13771

[30] Lepeng Zhao, Zhenhua Zou, Shuo Li, and Zhuotao Liu. 2026. Anonymization-Enhanced Privacy Protection for Mobile GUI Agents: Available but Invisible. *arXiv preprint arXiv:2602.10139* (2026). doi:10.48550/arXiv.2602.10139